C++ STL 算法竞赛速查大全

面向 ACM / ICPC / 蓝桥杯 / Codeforces / LeetCode 等

目标：种类尽量全，说明尽量短，查起来尽量快。

覆盖：容器、迭代器、算法、字符串、数值工具、位运算、函数对象、C++17/20 常用功能，以及竞赛常用 STL 模板。

建议比赛前重点熟悉：vector / string / pair / set / map / unordered_map / stack / queue / priority_queue / sort / lower_bound / next_permutation / accumulate / bitset。

目录

1. 万能头文件与基础习惯

2. vector / array / deque / list

3. pair / tuple

4. set / multiset / unordered_set

5. map / multimap / unordered_map

6. stack / queue / priority_queue

7. string

8. bitset

9. 迭代器与范围

10. 排序、查找与二分

11. 修改、统计、集合算法

12. 排列组合与堆算法

13. 数值算法与数学工具

14. lambda / function / comparator

15. C++17/20 竞赛常用

16. PBDS（GNU 扩展）

17. 常见容器选择表

18. 高频竞赛模板

1. 万能头文件与基础习惯

竞赛基本模板

GNU C++ 环境下最常见的开头。

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using pii = pair<int,int>;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    return 0;
}
```

竞赛提示： `bits/stdc++.h` 是 GNU 扩展，不属于 ISO C++ 标准，但多数算法竞赛平台支持。`ios::sync_with_stdio(false); cin.tie(nullptr);` 用于加速 cin/cout。

2. 顺序容器

vector - 动态数组

竞赛最常用容器。连续内存、随机访问快、尾部增删快。

操作 / 写法	作用	复杂度 / 备注
vector<int> a;	创建空 vector	-
vector<int> a(n);	创建 n 个元素，默认初始化	O(n)
vector<int> a(n, x);	n 个元素，值均为 x	O(n)
a.push_back(x)	尾部加入 x	均摊 O(1)
a.emplace_back(args...)	原地构造尾部元素	均摊 O(1)
a.pop_back()	删除最后元素	O(1)
a[i] / a.at(i)	访问第 i 个元素	O(1)；at 越界检查
a.front() / a.back()	首 / 尾元素	O(1)
a.size() / a.empty()	大小 / 是否为空	O(1)
a.clear()	清空元素	O(n)
a.resize(n, x)	调整大小；新元素可填 x	通常 O(n)
a.reserve(n)	预留容量，不改变 size	减少扩容
a.insert(it, x)	在 it 前插入	O(n)
a.erase(it)	删除 it 指向元素	O(n)
a.erase(l, r)	删除 [l,r)	O(n)
sort(a.begin(),a.end())	升序排序	O(n log n)
reverse(a.begin(),a.end())	反转	O(n)

```
vector<int> a = {3,1,4};
for (int x : a) cout << x << ' ';
sort(a.begin(), a.end(), greater<int>()); // ■■
```

竞赛提示： 需要频繁中间插入/删除时 vector 不一定合适；但竞赛中即使如此，也常因缓存友好而优先考虑 vector。

array - 定长数组

固定长度的 STL 数组，支持迭代器和 STL 算法。

操作 / 写法	作用	复杂度 / 备注
array<int,5> a{};	5 个 int，零初始化	栈上定长
a.fill(x)	全部填为 x	O(N)
a.size()	固定大小 N	O(1)
a.begin()/a.end()	迭代器	可直接 sort

deque - 双端队列

两端插入删除都快，也支持随机访问。

操作 / 写法	作用	复杂度 / 备注
d.push_back(x) / push_front(x)	尾 / 头加入	O(1)
d.pop_back() / pop_front()	尾 / 头删除	O(1)
d[i]	随机访问	O(1)
d.front()/d.back()	访问两端	O(1)

> 竞赛提示： 滑动窗口、单调队列通常直接用 deque。

list / forward_list

双向链表 / 单向链表。竞赛较少使用。

操作 / 写法	作用	复杂度 / 备注
lst.push_front(x)	头插	O(1)
lst.push_back(x)	尾插（list）	O(1)
lst.insert(it,x)	指定位置插入	O(1)，已知迭代器
lst.erase(it)	指定位置删除	O(1)
lst.sort()	链表自身排序	O(n log n)

> 竞赛提示： 不能随机访问；很多题 vector/deque 更实用。

3. pair 与 tuple

pair

把两个值绑在一起；图论、坐标、排序键极常见。

操作 / 写法	作用	复杂度 / 备注
pair<int,int> p={1,2};	创建 pair	-
p.first / p.second	访问两个成员	O(1)
make_pair(a,b)	创建 pair	通常可直接 {a,b}
tie(x,y)=p	拆包	-
swap(p1,p2)	交换	-

```
vector<pair<int,int>> v = {{2,3},{1,9},{1,4}};
sort(v.begin(), v.end());
// pair ■■■■■ first■■■■ second
```

tuple

可保存 3 个及以上不同类型的值。

操作 / 写法	作用	复杂度 / 备注
tuple<int,int,string> t;	创建 tuple	-
get<0>(t)	取第 0 个元素	编译期索引

操作 / 写法	作用	复杂度 / 备注
auto [a,b,c]=t;	结构化绑定	C++17
tie(a,b,c)=t;	拆包	-

4. 集合类容器

set - 有序去重集合

红黑树实现；自动排序、元素唯一。

操作 / 写法	作用	复杂度 / 备注
set<int> s;	创建	-
s.insert(x)	插入	O(log n)
s.erase(x)	按值删除	O(log n)
s.find(x)	查找；不存在返回 end	O(log n)
s.count(x)	是否存在（0/1）	O(log n)
s.lower_bound(x)	第一个 >= x	O(log n)
s.upper_bound(x)	第一个 > x	O(log n)
s.begin()/rbegin()	最小 / 最大元素位置	O(1)

```
set<int> s = {5,1,3};
if (auto it = s.lower_bound(2); it != s.end())
    cout << *it; // 3
```

multiset - 有序可重复集合

与 set 类似，但允许重复值。

操作 / 写法	作用	复杂度 / 备注
ms.insert(x)	插入一个 x	O(log n)
ms.count(x)	x 的数量	O(log n)+结果数
ms.erase(x)	删除所有 x	O(log n)+删除数
ms.erase(ms.find(x))	只删除一个 x	O(log n)

竞赛提示： 只删一个重复元素时不要直接 `erase(x)`。

unordered_set

哈希集合；无序，平均查找更快。

操作 / 写法	作用	复杂度 / 备注
us.insert(x)	插入	平均 O(1)
us.find(x)	查找	平均 O(1)
us.erase(x)	删除	平均 O(1)
us.count(x)	是否存在	平均 O(1)

竞赛提示： 最坏可退化到 O(n)；需要有序、lower_bound 或稳定复杂度时用 set。

5. 映射类容器

map - 有序键值表

键唯一且自动按 key 排序。

操作 / 写法	作用	复杂度 / 备注
map<string,int> mp;	创建	-
mp[k]	访问；不存在会创建默认值	O(log n)
mp.at(k)	访问；不存在抛异常	O(log n)
mp[k]=v	插入/修改	O(log n)

操作 / 写法	作用	复杂度 / 备注
mp.find(k)	查找	O(log n)
mp.count(k)	是否存在	O(log n)
mp.erase(k)	删除	O(log n)
mp.lower_bound(k)	第一个 key >= k	O(log n)

```
map<string,int> cnt;
for (string x : a) cnt[x]++;
for (auto &[key, val] : cnt)
    cout << key << ' ' << val << '\n';
```

unordered_map

哈希映射；计数、映射时非常常用。

操作 / 写法	作用	复杂度 / 备注
mp[k]++	频率统计	平均 O(1)
mp.find(k)	查找	平均 O(1)
mp.contains(k)	是否包含 key	C++20，平均 O(1)
mp.reserve(n)	预留桶空间	可减少 rehash

竞赛提示： 自定义类型作为 key 时需要提供 hash。对抗性数据下可能被卡哈希。

multimap

允许相同 key 的有序映射。

操作 / 写法	作用	复杂度 / 备注
mm.insert({k,v})	插入	O(log n)
mm.equal_range(k)	得到所有 key=k 的区间	O(log n)+结果数

6. 容器适配器

stack - 栈

LIFO：后进先出。DFS、括号匹配、单调栈常用。

操作 / 写法	作用	复杂度 / 备注
st.push(x)	入栈	O(1)
st.pop()	弹栈	O(1)
st.top()	栈顶	O(1)
st.empty()/size()	判空 / 大小	O(1)

queue - 队列

FIFO：先进先出。BFS 常用。

操作 / 写法	作用	复杂度 / 备注
q.push(x)	入队	O(1)
q.pop()	出队	O(1)
q.front()/back()	队首 / 队尾	O(1)

priority_queue - 优先队列 / 堆

默认大根堆；Dijkstra、Top-K、贪心经常使用。

操作 / 写法	作用	复杂度 / 备注
priority_queue<int> q;	大根堆	top 最大
priority_queue<int,vector<int>,greater<int>> q;	小根堆	top 最小
q.push(x)	加入	O(log n)
q.pop()	删除堆顶	O(log n)
q.top()	访问堆顶	O(1)

```
using P = pair<int,int>;
priority_queue<P, vector<P>, greater<P>> pq;
pq.push({dist, node});
```

7. string

string - 字符串

本质上是可变长字符数组，支持大量实用操作。

操作 / 写法	作用	复杂度 / 备注
string s; / string s(n,'a');	创建字符串	-
s.size()/length()	长度	O(1)
s[i] / s.at(i)	访问字符	O(1)
s += t / s.append(t)	拼接	均摊与追加长度相关
s.push_back(c)	尾部加字符	均摊 O(1)
s.pop_back()	删末字符	O(1)
s.substr(pos,len)	取子串	O(len)
s.find(t,pos)	查找子串	返回位置或 string::npos
s.rfind(t)	从后查找	-
s.insert(pos,t)	插入	O(n)
s.erase(pos,len)	删除	O(n)
s.replace(pos,len,t)	替换	O(n)
stoi/stoll/stod(s)	字符串转数字	-
to_string(x)	数字转字符串	-

```
string s = "abcdef";
cout << s.substr(2, 3); // cde
if (s.find("cd") != string::npos) { /* found */ }
sort(s.begin(), s.end());
reverse(s.begin(), s.end());
```

字符判断与转换

处理单个字符。

操作 / 写法	作用	复杂度 / 备注
isdigit(c)	是否数字	-
isalpha(c)	是否字母	-
isalnum(c)	是否字母或数字	-
islower(c)/isupper(c)	大小写判断	-
tolower(c)/toupper(c)	大小写转换	-

8. bitset

bitset - 固定位集合

高效存储/批量处理布尔位；状态压缩、集合运算常用。

操作 / 写法	作用	复杂度 / 备注
bitset<1000> b;	1000 位，初始 0	编译期固定大小
b.set(i) / reset(i) / flip(i)	置1 / 置0 / 翻转	O(1)
b[i] / b.test(i)	读某位	O(1)
b.count()	1 的个数	很快
b.any()/none()/all()	任意1 / 全0 / 全1	-
b << k / b >> k	整体移位	-

操作 / 写法	作用	复杂度 / 备注
b1 & b2 / | / ^ / ~	位集合运算	-
b.to_ulong()	转无符号整数	注意范围
b.to_string()	转 01 字符串	O(N)

竞赛提示： 大量布尔 DP / 可达性问题中， bitset 常能把按位循环优化成机器字并行。

9. 迭代器与范围

iterator

STL 算法通常使用左闭右开区间 `[first,last)`。

操作 / 写法	作用	复杂度 / 备注
a.begin()/a.end()	首元素 / 尾后位置	-
a.rbegin()/a.rend()	反向迭代器	-
next(it,k)	向后移动 k	复杂度依迭代器
prev(it,k)	向前移动 k	-
advance(it,k)	原地移动迭代器	-
distance(l,r)	迭代器距离	随机访问 O(1)，否则 O(n)

```
for (auto it = s.begin(); it != s.end(); ++it) cout << *it;
for (auto it = s.rbegin(); it != s.rend(); ++it) cout << *it;
```

10. 排序、查找与二分

排序

最常用算法之一。

操作 / 写法	作用	复杂度 / 备注
sort(l,r)	升序排序	O(n log n)
sort(l,r,greater<T>())	降序排序	O(n log n)
stable_sort(l,r)	稳定排序	O(n log n)
partial_sort(l,m,r)	只保证前 m-l 个最小且有序	O(n log k)
nth_element(l,nth,r)	让 nth 位成为排序后应有元素	平均 O(n)

```
sort(v.begin(), v.end(), [](const auto& a, const auto& b) {
    if (a.second != b.second) return a.second > b.second;
    return a.first < b.first;
});
```

二分与查找

要求区间已按相同规则有序。

操作 / 写法	作用	复杂度 / 备注
lower_bound(l,r,x)	第一个 >= x	O(log n) 随机访问
upper_bound(l,r,x)	第一个 > x	O(log n)
binary_search(l,r,x)	是否存在 x	O(log n)
equal_range(l,r,x)	返回 [lower,upper)	O(log n)
find(l,r,x)	线性找 x	O(n)
find_if(l,r,pred)	找第一个满足条件	O(n)

```
int pos = lower_bound(a.begin(), a.end(), x) - a.begin();
int cnt = upper_bound(a.begin(),a.end(),x)
```

```
    - lower_bound(a.begin(),a.end(),x);
```

11. 修改、统计、集合算法

统计与判断

对区间做常见检查。

操作 / 写法	作用	复杂度 / 备注
count(l,r,x)	统计 x 次数	O(n)
count_if(l,r,pred)	统计满足条件元素	O(n)
all_of(l,r,pred)	是否全部满足	O(n)
any_of(l,r,pred)	是否至少一个满足	O(n)
none_of(l,r,pred)	是否都不满足	O(n)
min_element(l,r)	最小元素迭代器	O(n)
max_element(l,r)	最大元素迭代器	O(n)
minmax_element(l,r)	同时找最小最大	O(n)

修改区间

批量修改元素。

操作 / 写法	作用	复杂度 / 备注
fill(l,r,x)	全部赋值 x	O(n)
replace(l,r,a,b)	把 a 替换为 b	O(n)
reverse(l,r)	反转	O(n)
rotate(l,m,r)	旋转，使 m 成为开头	O(n)
swap(a,b)	交换	通常 O(1)
transform(l,r,out,f)	逐元素变换	O(n)
copy(l,r,out)	复制区间	O(n)

去重 erase-unique

vector 排序去重的经典写法。

```
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());
```

竞赛提示：`unique` 只把相邻重复元素移到后面，并返回新逻辑结尾；它本身不会缩短 vector。

有序集合算法

输入必须有序，输出写入目标迭代器。

操作 / 写法	作用	复杂度 / 备注
set_union	并集	O(n+m)
set_intersection	交集	O(n+m)
set_difference	差集 A-B	O(n+m)
set_symmetric_difference	对称差	O(n+m)
includes	A 是否包含 B	O(n+m)

```
vector<int> c;
set_intersection(a.begin(),a.end(),b.begin(),b.end(),
                 back_inserter(c));
```

12. 排列组合与堆算法

next_permutation / prev_permutation

按字典序生成下一个 / 上一个排列。

操作 / 写法	作用	复杂度 / 备注
next_permutation(l,r)	下一个排列；无则回到最小并返回 false	O(n)
prev_permutation(l,r)	上一个排列	O(n)

```
sort(a.begin(), a.end());
do {
    // ■■■■■■
} while (next_permutation(a.begin(), a.end()));
```

堆算法

直接把 vector 的一段维护成堆。priority_queue 更常用。

操作 / 写法	作用	复杂度 / 备注
make_heap(l,r)	建大根堆	O(n)
push_heap(l,r)	最后元素加入堆	O(log n)
pop_heap(l,r)	堆顶移到末尾	O(log n)
sort_heap(l,r)	堆排序	O(n log n)

13. 数值算法与数学工具

常见数值区间算法。

操作 / 写法	作用	复杂度 / 备注
accumulate(l,r,init)	求和 / 累积	O(n)
iota(l,r,start)	填连续整数	O(n)
partial_sum(l,r,out)	前缀和	O(n)
adjacent_difference(l,r,out)	相邻差分	O(n)
inner_product(...)	内积	O(n)
gcd(a,b)	最大公约数,	O(log min)
lcm(a,b)	最小公倍数,	注意溢出

```
long long sum = accumulate(a.begin(), a.end(), 0LL);
vector<int> id(n);
iota(id.begin(), id.end(), 0);
long long g = gcd(x, y);
```

常用

浮点、幂、取整等。

操作 / 写法	作用	复杂度 / 备注
sqrt(x)	平方根	浮点
pow(a,b)	a^b	浮点；整数快速幂通常自己写
abs(x)	绝对值	-
ceil/floor/round(x)	向上 / 向下 / 四舍五入	-
log/log2/log10	对数	浮点
sin/cos/atan2	三角函数	几何题

竞赛提示： 整数幂、模幂不要依赖 `pow`，应写快速幂避免精度问题。

14. lambda / function / comparator

Lambda 表达式

竞赛里自定义排序、局部 DFS、谓词非常方便。

```
auto cmp = [](const pair<int,int>& a, const pair<int,int>& b) {
    return a.second < b.second;
};

sort(v.begin(), v.end(), cmp);

auto is_even = [](int x) { return x % 2 == 0; };
int cnt = count_if(a.begin(), a.end(), is_even);
```

竞赛提示： 排序比较器应表达 “ a 是否严格排在 b 前面 ”， 不要写 `<=`，否则违反 strict weak ordering。

function

保存可调用对象；递归 lambda 常见，但有额外开销。

```
function<void(int,int)> dfs = [&](int u, int p) {
    for (int v : g[u])
        if (v != p) dfs(v, u);
};
```

15. C++17 / C++20 竞赛常用

结构化绑定

拆 pair / tuple / map 元素。

```
auto [x, y] = p;
for (auto &[key, val] : mp) val++;
```

if 初始化语句

缩短查找代码。

```
if (auto it = mp.find(x); it != mp.end()) {
    cout << it->second;
}
```

std::optional（较少）

表示“ 可能有值 ”。

操作 / 写法	作用	复杂度 / 备注
optional<int> x;	空 optional	C++17
x.has_value() / if(x)	是否有值	-
*x / x.value()	取值	-

contains

C++20 中 set/map/unordered_* 可直接判断键是否存在。

```
if (mp.contains(key)) { ... }
if (s.contains(x)) { ... }
```

ranges（了解）

C++20 ranges 可减少 begin/end，但竞赛平台标准版本不总统一。

```
// C++20
ranges::sort(a);
auto it = ranges::lower_bound(a, x);
```

竞赛提示： 为了兼容大多数评测环境，传统 `sort(a.begin(), a.end())` 仍是最稳妥写法。

16. PBDS（GNU 扩展）

ordered_set

支持“ 第 k 小 ”和“ 某值排名 ”的平衡树；不是标准 STL，但竞赛很有用。

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template<class T>
using ordered_set = tree<
    T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

ordered_set<int> s;
s.insert(10);
s.insert(20);

auto it = s.find_by_order(0); // ■ 0 ■
int k = s.order_of_key(20);   // ■■■■ 20 ■■■■
```

竞赛提示： 若要支持重复值，常存 `pair`。PBDS 依赖 GNU 编译器。

17. 常见容器选择表

需求	首选	原因
动态数组、下标访问	vector	最常用，随机访问 O(1)
固定长度数组	array	STL 化的定长数组
两端频繁增删	deque	两端 O(1)
LIFO	stack	栈
FIFO / BFS	queue	队列
随时取最大/最小	priority_queue	堆顶 O(1)
有序且去重	set	O(log n)，支持 lower_bound
有序且允许重复	multiset	重复元素
key -> value 且有序	map	O(log n)
快速查存在/计数	unordered_set/map	平均 O(1)
固定大量布尔位	bitset	内存小、位并行
第 k 小 / 排名	PBDS ordered_set	order statistics

18. 高频竞赛模板

排序 + 去重

```
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());
```

离散化

```
vector<int> xs = a;
sort(xs.begin(), xs.end());
xs.erase(unique(xs.begin(), xs.end()), xs.end());

int id = lower_bound(xs.begin(), xs.end(), x) - xs.begin();
```

频率统计

```
unordered_map<int,int> cnt;
for (int x : a) cnt[x]++;
```

Top-K 最小元素（维护大根堆）

```
priority_queue<int> pq;
for (int x : a) {
    pq.push(x);
    if ((int)pq.size() > k) pq.pop();
}
// pq.top() ■■■ k ■■■■■■■■■■
```

BFS

```
queue<int> q;
vector<int> dist(n, -1);
dist[s] = 0;
q.push(s);

while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : g[u]) {
        if (dist[v] == -1) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}
```

Dijkstra

```
const long long INF = (1LL<<60);
vector<long long> d(n, INF);
priority_queue<pair<long long,int>,
               vector<pair<long long,int>>,
               greater<pair<long long,int>>> pq;

d[s] = 0;
pq.push({0, s});

while (!pq.empty()) {
    auto [du, u] = pq.top(); pq.pop();
    if (du != d[u]) continue;

    for (auto [v, w] : g[u]) {
        if (d[v] > du + w) {
            d[v] = du + w;
            pq.push({d[v], v});
        }
    }
}
```

单调队列：滑动窗口最小值核心

```
deque<int> q; // ■■■
for (int i = 0; i < n; ++i) {
    while (!q.empty() && q.front() <= i-k) q.pop_front();
    while (!q.empty() && a[q.back()] >= a[i]) q.pop_back();
    q.push_back(i);
    if (i >= k-1) cout << a[q.front()] << ' ';
}
```

二分答案骨架

```
long long l = 0, r = LIMIT;
while (l < r) {
    long long mid = l + (r-l)/2;
    if (check(mid)) r = mid;
    else l = mid + 1;
}
cout << l;
```

自定义结构排序

```
struct Node {
    int x, y;
};

sort(v.begin(), v.end(), [](const Node& a, const Node& b) {
    if (a.x != b.x) return a.x < b.x;
    return a.y > b.y;
});
```

最后速记

最需要形成肌肉记忆的 12 个：
vector、pair、set、map、unordered_map、stack、queue、priority_queue、sort、lower_bound、unique、accumulate。

复杂度直觉： vector 下标 O(1)；set/map O(log n)；unordered_* 平均 O(1)；priority_queue push/pop O(log n)；sort O(n log n)；线性算法通常 O(n)。

区间规则： STL 几乎统一使用左闭右开 `[begin,end)`。

类型安全： 求和、距离、权值很大时优先 `long long`；`accumulate` 的初值写 `0LL` 可避免 int 溢出。

—— C++ STL 算法竞赛速查大全 · 精简版 ——